Chapter 53: Profiling Angular Applications

1. Overview

Profiling is the discipline of turning "the app feels slow" into a specific, falsifiable claim: *this component's `ngOnChanges` runs 340 times per keystroke*, *this bundle chunk is 1.2 MB because of a moment.js locale import*, *LCP is 4.1s because the hero image is discovered late by the preload scanner*. Staff-level engineers are expected to diagnose performance regressions without guessing — using Angular DevTools, the Chrome Performance panel, Core Web Vitals field/lab data, Lighthouse CI gating, and bundle analyzers, then translate findings into a concrete fix (usually `OnPush`, `trackBy`, signal-based reactivity, code-splitting, or removing an accidental subscription/re-render loop).

This chapter treats profiling as an end-to-end workflow: **reproduce → capture a trace → localize the hot path → form a hypothesis → fix → re-measure → gate in CI so it doesn't regress**. Skipping straight to "add `OnPush` everywhere" without measurement is exactly the anti-pattern interviewers probe for.

---

2. Core Concepts

2.1 Angular DevTools Profiler

Angular DevTools (the browser extension) has two panes relevant here: the **Components** tree (inspect inputs/outputs/state live) and the **Profiler**.

- **What it records**: one bar per change-detection (CD) cycle. Angular runs CD as a synchronous, depth-first pass over the component tree (or, with signals, a more surgical rerun) triggered by zone.js patching an async API (click, `setTimeout`, XHR/fetch, etc.), or manually via `ApplicationRef.tick()` / `ChangeDetectorRef.detectChanges()`.
- **Bar chart view**: X-axis is time, each bar is one CD cycle; bar height/color encodes duration. A cluster of tall bars in rapid succession usually means an event handler (e.g., a `mousemove` or `scroll` listener) is triggering CD far more often than needed.
- **Flame-graph-per-cycle view**: click a bar to see a per-component breakdown of that single cycle — which components were **checked**, how long each check took, and (importantly) whether the check actually caused a DOM write or was a no-op "nothing changed" traversal.
- **"Change detection wasn't run"** hierarchy: for `OnPush` components, DevTools shows greyed-out/skipped subtrees, which is the fastest visual way to confirm your `OnPush` boundary is actually working (versus the parent still passing a new object reference every cycle, defeating the optimization).
- Angular DevTools profiler also flags **hydration** issues in SSR apps and shows the **source of a CD trigger** in recent versions (event name, e.g. `click`, or a zone-less signal write).

Key interview point: the profiler measures *how many times a component's template check function ran and how long each run took*, not raw rendering cost. A component can be "checked" (CPU cost) without producing any DOM mutation.

2.2 Chrome Performance Tab

This is the ground truth for **all** work the main thread does — not just Angular's. Recording a trace during the same interaction gives you:

- **Flame chart (Main track)**: nested call stacks over time. Angular-specific frames appear as `Zone.runTask`, `ApplicationRef.tick`, `ChangeDetectorRef.detectChanges`, `ComponentFactory.create`, `RouterOutlet`, etc. You can see zone.js's monkey-patched task boundaries wrapping the actual CD call.
- **Long tasks**: any task on the main thread > 50ms is flagged with a red triangle. Long tasks are the direct cause of poor **INP** (Interaction to Next Paint) because they block the browser from responding to input.
- **Bottom-Up / Call Tree / Event Log** tabs: aggregate self-time across the whole trace to find "who actually burned the CPU" — often a third-party script, a `JSON.parse` of a huge payload, or a synchronous layout thrash (`getBoundingClientRect` inside a loop causing forced reflow, visible as purple "Layout" / "Recalculate Style" blocks interleaved with scripting).
- **Rendering phases**: Scripting (yellow) → Rendering/Style/Layout (purple) → Painting/Compositing (green). A wide purple block right after a script block usually means the script mutated the DOM in a way that forced a synchronous layout (style/geometry read after a write, i.e. **layout thrashing**).
- **Screenshots + Frames track**: correlates dropped frames with what the main thread was doing at that moment — the direct visual evidence for jank.

Angular-specific correlation trick: because zone.js wraps every async callback in `NgZone.runTask`, you can search/filter the flame chart for `zone.js` frames to isolate exactly which Angular event handler is responsible for a given long task, then drill into its children to see which component/service call dominates self-time.

2.3 Core Web Vitals and How Angular Apps Affect Them

Metric	What it measures	Angular-specific contributors
LCP (Largest Contentful Paint)	Time until the largest visible element (usually hero image/text block) renders	Large initial JS bundle blocking hydration/rendering; client-side-rendered hero content (no SSR); render-blocking global CSS; slow API call gating the hero content; images not preloaded/not using `NgOptimizedImage`
INP (Interaction to Next Paint, replaced FID in March 2024)	Worst-case latency between a user interaction and the next visual update	Long CD cycles (huge component trees checked on every keystroke), heavy synchronous work in event handlers, zone.js triggering CD for unrelated async work, unthrottled `scroll` / `input` listeners
CLS (Cumulative Layout Shift)	Unexpected layout movement	Images/ads without reserved dimensions, late-injected banners, web fonts causing FOUT/FOIT reflow, `*ngIf`-toggled content pushing the page before data arrives

Angular-specific levers:

- **SSR + hydration** (Angular Universal / `provideClientHydration`) improves LCP by shipping meaningful

HTML immediately, but a hydration mismatch or "destructive hydration" (pre-hydration re-render) can hurt both LCP and CLS if it repaints the shell.

- **Zoneless Angular** (`provideZonelessChangeDetection`, stable direction since Angular 18+) reduces INP risk because CD is no longer triggered speculatively for every possible async source — only for actual signal writes / explicit markForCheck.
- **Deferred loading (`@defer`)** directly targets LCP/INP by removing below-the-fold or interaction-gated component code from the initial bundle and CD graph.
- `NgOptimizedImage` targets LCP by adding `fetchpriority="high"`, enforcing width/height (helps CLS too), and integrating with image CDNs.

2.4 Lighthouse CI Integration

Lighthouse gives **lab data** (synthetic, reproducible) versus Chrome UX Report / RUM giving **field data** (real users, real network/device variance). Both matter; lab data is what you gate CI on because it's deterministic.

- `@lhci/cli` runs Lighthouse against a built app N times (median run reduces noise), asserts against a config of budgets (`lighthouserc.json`), and fails the build if a metric regresses past a threshold.
- Typical Angular CI pipeline: `ng build --configuration production` → serve the `dist/` output with a static server → `lhci autorun` against that URL → assert on `categories:performance`, `first-contentful-paint`, `largest-contentful-paint`, `total-blocking-time`, `cumulative-layout-shift`, and a JS **bundle-size budget**.
- Because Lighthouse itself introduces variance (CPU throttling emulation, background CI runner noise), best practice is `numberOfRuns: 3-5` with median aggregation, and comparing against a **stored baseline** rather than an absolute number, so pipelines catch regressions (e.g., "+15% TBT vs. last main") rather than chasing an arbitrary absolute score.

2.5 Bundle Analysis

- `source-map-explorer`: takes the built JS + its `.map` files and renders a treemap of *which source module contributes how many bytes* to each output chunk. Requires `"sourceMap": true` in `angular.json`'s production config (or `--source-map` flag) temporarily.
- `webpack-bundle-analyzer`: works if you have access to the underlying webpack stats (Angular's builder wraps webpack/esbuild); for esbuild-based `@angular-devkit/build-angular:application` builder, `esbuild`'s own `--metafile` + `esbuild-visualizer` is the modern equivalent.
- What you're hunting for: duplicate library versions (two RxJS or two lodash copies because of a version mismatch), accidentally-imported full libraries (`import _ from 'lodash'` instead of `import debounce from 'lodash/debounce'`), locale data bloat (`@angular/common/locales/all`), polyfills that are no longer needed for the target browserslist, and eagerly-loaded feature modules that should be lazy (`loadChildren`) or `@defer` red.
- Angular CLI's built-in `ng build --stats-json` (esbuild) or the **bundle budgets** in `angular.json` (`budgets: [{ type: "initial", maximumWarning: "500kb", maximumError: "1mb" }]`) are the first line of defense — they fail the build before you even need a visual tool.

2.6 Memory Profiling (Heap Snapshots)

- Chrome DevTools **Memory** panel → **Heap snapshot**: a point-in-time graph of every JS object and its

retainers. Take one snapshot, perform an action N times (e.g., open/close a modal 10 times), force GC, take a second snapshot, and use **Comparison view** to see objects whose count grew by N (or a multiple) — a strong signal of a leak.

- Common Angular leak sources:
  - Subscribing to an `Observable` (a service-level `Subject`, `router.events`, a `fromEvent` on `window`) in a component without unsubscribing (`takeUntilDestroyed()`, `async` pipe, or manual `Subscription` teardown in `ngOnDestroy`).
  - `@HostListener` on `window`/`document` bound in a component that's destroyed, if not cleaned up (Angular removes host listeners it created automatically, but manually-added `addEventListener` calls are not).
  - Detached DOM trees: closures or component references held by a service/singleton keep a whole component (and its DOM) alive after Angular has logically destroyed it — visible in the heap snapshot as "Detached HTMLElement" nodes with a nonzero retainer count.
  - Third-party widgets (maps, charts) that need explicit `.destroy()` calls not wired into `ngOnDestroy`.
- **Allocation instrumentation timeline** (record while interacting) shows *when* allocations spike, useful for correlating a leak with a specific user action rather than diffing two static snapshots.
- **Performance panel's memory checkbox** overlays a JS heap size graph on the same timeline as CPU activity — a sawtooth that returns to baseline after each GC is healthy; a staircase that keeps climbing is a leak.

---

3. Code Examples

3.1 Scenario: excess CD cycles from object identity + missing `trackBy`

```
// dashboard.component.ts  -- BEFORE: causes excessive CD work
import { Component, Input } from '@angular/core';

interface Metric {
  id: string;
  label: string;
  value: number;
}

@Component({
  selector: 'app-dashboard',
  standalone: true,
  template: `
    <input (input)="onFilter($event)" placeholder="Filter metrics..." />

    <!-- Problem 1: getFilteredMetrics() is called on every CD cycle for -->
    <!-- EVERY keystroke, and it allocates a brand-new array each time. -->
    <app-metric-row
      *ngFor="let m of getFilteredMetrics()"
      [metric]="m">
```

```
    </app-metric-row>
  `,
})
export class DashboardComponent {
  @Input() metrics: Metric[] = [];
  private filterText = '';

  onFilter(e: Event) {
    this.filterText = (e.target as HTMLInputElement).value;
  }

  // Recomputed + re-allocated on EVERY change detection pass, not just
  // when `metrics` or `filterText` actually change. Default (non-OnPush)
  // strategy also means every ancestor CD cycle re-checks this component
  // and re-runs this function, then diffs the (new-identity) array via
  // NgForOf, which — because item identity changed — tears down and
  // rebuilds every <app-metric-row>, re-running ITS change detection,
  // re-creating DOM nodes, and losing any local state (e.g., animations,
  // focus, scroll position) in the row components.
  getFilteredMetrics(): Metric[] {
    return this.metrics.filter(m =>
      m.label.toLowerCase().includes(this.filterText.toLowerCase())
    );
  }
}

@Component({
  selector: 'app-metric-row',
  standalone: true,
  template: `<div>{{ metric.label }}: {{ metric.value }}</div>`,
})
export class MetricRowComponent {
  @Input() metric!: Metric;
  // Default ChangeDetectionStrategy -> re-checked on every ancestor tick
  // regardless of whether `metric` changed.
}
```

In Angular DevTools' profiler, this shows up as: a bar per keystroke, each containing hundreds of `MetricRowComponent` checks even when only the filter text changed and the underlying `metrics` array did not — the "damage" is the array re-allocation defeating `NgForOf`'s default identity tracking, compounded by every row being on the default strategy.

```
// dashboard.component.ts  -- AFTER: OnPush + trackBy + memoized derivation
import { Component, Input, ChangeDetectionStrategy } from '@angular/core';

interface Metric {
  id: string;
  label: string;
  value: number;
}

@Component({
```

```
  selector: 'app-dashboard',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <input (input)="onFilter($event)" placeholder="Filter metrics..." />

    <app-metric-row
      *ngFor="let m of filteredMetrics; trackBy: trackById"
      [metric]="m">
    </app-metric-row>
  `,
})
export class DashboardComponent {
  @Input() metrics: Metric[] = [];
  filteredMetrics: Metric[] = [];
  private filterText = '';

  ngOnChanges() {
    this.recompute();
  }

  onFilter(e: Event) {
    this.filterText = (e.target as HTMLInputElement).value.toLowerCase();
    this.recompute(); // explicit, on the actual trigger — not every CD tick
  }

  // Computed once per real change, not once per CD cycle.
  private recompute() {
    this.filteredMetrics = this.metrics.filter(m =>
      m.label.toLowerCase().includes(this.filterText)
    );
  }

  // Stable identity function: NgForOf reuses existing MetricRowComponent
  // instances (and their DOM nodes) instead of destroying/recreating them
  // whenever the array reference changes but the underlying items don't.
  trackById(_index: number, metric: Metric): string {
    return metric.id;
  }
}

@Component({
  selector: 'app-metric-row',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<div>{{ metric.label }}: {{ metric.value }}</div>`,
})
export class MetricRowComponent {
  @Input() metric!: Metric;
  // OnPush: only re-checked when `metric`'s reference changes (or an
  // event originates within it, or markForCheck/async pipe fires) --
  // Angular skips the whole subtree otherwise, visible as greyed-out
  // bars in the DevTools profiler.
```

```
}
```

Signals-based version (Angular 17+, removes the manual `recompute()` / `ngOnChanges` bookkeeping entirely and is inherently push-based):

```
import { Component, input, signal, computed, ChangeDetectionStrategy } from '@angular/core';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <input (input)="filterText.set($any($event.target).value.toLowerCase())" />
    <app-metric-row *ngFor="let m of filteredMetrics(); trackBy: trackById" [metric]="m" />
  `,
})
export class DashboardComponent {
  metrics = input<Metric[]>([]);
  filterText = signal('');

  // Recomputes lazily, only when `metrics` or `filterText` actually change,
  // and only re-renders the template if the computed *value* differs.
  filteredMetrics = computed(() =>
    this.metrics().filter(m => m.label.toLowerCase().includes(this.filterText()))
  );

  trackById(_i: number, m: Metric) {
    return m.id;
  }
}
```

3.2 Bundle analysis invocation

```
# 1. Build production bundle WITH source maps (temporarily) so the
#    analyzer can map minified bytes back to source modules.
ng build --configuration production --source-map

# 2. Run source-map-explorer against the main output chunk(s).
#    Generates an interactive treemap (HTML) sized by gzip bytes.
npx source-map-explorer dist/my-app/browser/main-*.js --html bundle-report.html

# 3. Analyze every chunk at once (useful for spotting duplicated deps
#    across lazy chunks, e.g. two copies of rxjs or moment).
npx source-map-explorer dist/my-app/browser/*.js --html bundle-report-all.html

# 4. Modern esbuild-based Angular builder: use the metafile directly.
ng build --configuration production --stats-json
npx esbuild-visualizer --metadata dist/my-app/stats.json --filename stats.html

# 5. CI budget gate (fails build over threshold) -- angular.json:
#    "budgets": [
```

```
#       { "type": "initial", "maximumWarning": "500kb", "maximumError": "1mb" },
#       { "type": "anyComponentStyle", "maximumWarning": "6kb" }
#     ]

# 6. Lighthouse CI against the built, statically-served app.
npm install -D @lhci/cli
npx http-server dist/my-app/browser -p 4200 &
npx lhci autorun --collect.url=http://localhost:4200 --collect.numberOfRuns=5
```

---

4. Internal Working

4.1 How the DevTools profiler times each component's check

Every Angular component's compiled template produces an **update function** invoked during change detection (`ɵɵadvance`, `ɵɵtextInterpolate`, property bindings, etc.), wrapped by the renderer's instrumentation. Angular DevTools hooks into `ApplicationRef.tick()` at the framework level:

1. When `ApplicationRef.tick()` runs (triggered by a zone `onMicrotaskEmpty` / `onStable`-adjacent hook, a signal effect scheduling a tick, or manual `detectChanges()`), the profiler's instrumentation wraps the traversal of the `LView` / `TView` tree.
2. For each component's `LView`, Angular calls its **template function** in "Update" mode (`RenderFlags.Update`). The profiler wraps that call with `performance.mark` / `now()` timestamps *before* and *after*, giving per-component wall-clock duration for that single cycle.
3. Angular's CD traversal is a depth-first walk of the component tree (a flattened array of `LView`s in practice, walking via `TNode` sibling/child pointers). For default-strategy components, every descendant's update function runs unconditionally. For `OnPush` components, the framework checks the component's `dirty` / `CheckAlways` flags first; if not dirty (no `@Input()` reference changed, no event originated inside it, no explicit `markForCheck()`), the update function call — and therefore that subtree — is skipped entirely, which is why DevTools renders it as a greyed-out node with ~0 duration.
4. The profiler aggregates these per-component timings into the bar you see for that tick, and additionally tags **why** the cycle started (in newer Angular DevTools versions) by reading the scheduler's recorded trigger — a DOM event name for zone-based CD, or the specific signal/effect for zoneless CD.
5. Because this instrumentation lives in dev-mode-only code paths (guarded by `ngDevMode` and the DevTools extension's injected hooks), it adds overhead — profiler numbers are always somewhat inflated versus an un-instrumented run, which is a real interview gotcha (see Section 5).

4.2 How Chrome's flame chart correlates JS execution with rendering/layout/paint

Chrome's rendering pipeline runs (conceptually) as: **Input** → **JavaScript** → **Style calculation** → **Layout** → **Paint** → **Composite**. The Performance panel's Main track renders this as a stack of colored blocks over a shared timeline:

- **Yellow blocks** = JS execution (call stack frames, nested by actual function calls — this is where you'll see `Zone.prototype.runTask`, `ApplicationRef.tick`, your component methods, RxJS operators, etc.).
- **Purple blocks** = rendering work: "Recalculate Style" (CSSOM matching) and "Layout" (geometry computation). These are placed on the timeline at the exact moment the browser actually performed

them — which, critically, can be interleaved *inside* a JS block if your code performs a **layout-forcing read** (e.g., `el.offsetHeight`, `getComputedStyle()`) immediately after a DOM write, forcing a synchronous "reflow" mid-script instead of letting the browser batch it at the natural end-of-frame point. This shows up as a narrow purple sliver nested inside a yellow block, and repeating this pattern in a loop ("layout thrashing") is a classic finding.

- **Green blocks** = Paint and Composite, generally happening after the script/layout work for that frame, on the same thread for paint (compositing itself may hand off to the compositor thread, visible as a separate track).
- **Frames track** (top) shows one tick per rendered frame with a thumbnail; a gap or a frame marked in red means a dropped/delayed frame, and you can hover to see exactly which JS/layout work on the Main track was still running when that frame should have been produced.
- **Bottom-Up view** re-aggregates the same call stacks by **total self time** across the whole recording regardless of when they occurred — this is how you find "which function/library is the actual CPU hog" rather than "what happened at time X," complementing the flame chart's chronological view.
- Zone.js's monkey patches (`zone.js` frames wrapping `addEventListener` callbacks, `setTimeout`, XHR `onreadystatechange`) appear explicitly in the flame chart, which is what lets you attribute a long task back to "this click handler triggered `ApplicationRef.tick()` which took 180ms checking 3,000 default-strategy components."

---

5. Edge Cases & Gotchas

- **Dev build vs. production build numbers are not comparable.** Dev builds include extra `ngDevMode` checks, un-minified/un-tree-shaken code, source-map overhead, and (with Angular DevTools attached) extra instrumentation hooks — all of which inflate CD and script timings relative to production. Always profile against a `ng build --configuration production` build served statically (or `ng serve --configuration production` — noting HMR/dev-server overhead can still differ from a real static host/CDN). A component that looks alarmingly slow in dev might be perfectly fine in prod, and conversely, aggressive dev-mode zone patching can *mask* an actual prod-only issue (e.g., differential minification breaking a fast-path).
- **CPU/network throttling matters more than the metric itself.** Lighthouse and Chrome's Performance panel both offer 4x/6x CPU slowdown and "Slow 4G" presets specifically because most real users are on far weaker hardware than developer machines. A trace recorded unthrottled on a dev laptop can show green Web Vitals while real users see red — always profile with throttling that approximates your actual traffic (check CrUX/RUM data for your real device/network mix before choosing a preset).
- **Over-indexing on one metric (usually LCP) while INP or CLS silently regress.** LCP is the most marketed metric and the easiest to chase (preload the hero image, done), but a team that ships a beautiful LCP number while leaving a 400ms input-to-paint delay on a search box or filter control will still fail Core Web Vitals field assessment (INP became a Core Web Vital in March 2024, replacing FID) and will still feel janky to users. Track all three together; don't let a single Lighthouse category score stand in for the full picture, and remember Lighthouse's lab TBT/TTI are *proxies* for INP, not INP itself — real INP requires field data (`web-vitals` JS library reporting to RUM, or CrUX).
- **Third-party scripts skew traces and get misattributed.** Tag managers, chat widgets, ad scripts, and analytics beacons routinely dominate a trace's long-task time and heap growth, but a naive read of "my app is slow" can lead you to blame your own components. Use the flame chart's Bottom-Up view and

filter/attribute by script URL (Chrome labels third-party origins), or use Lighthouse's "Reduce the impact of third-party code" audit, before concluding the regression is in your Angular code. Conversely, don't dismiss third-party impact either — it directly harms your real-user Web Vitals and Lighthouse score even though "you didn't write that code."

- `OnPush` **doesn't guarantee correctness-of-skip if inputs are mutated in place.** If a parent mutates an array/object passed as `@Input()` (`this.items.push(x)`) instead of replacing the reference, `OnPush` children never see a changed reference and are never re-checked — a silent stale-UI bug that looks like a profiling win (fewer CD cycles!) but is actually broken behavior. Profilers won't flag this; only manual testing or state-management discipline (immutable updates, signals) catches it.
- **Detached DOM in heap snapshots isn't always a real leak.** A "Detached HTMLElement" with zero or near-zero retaining paths that will be GC'd on the next cycle is normal; the interview-relevant skill is distinguishing a *transient* detached node (about to be collected) from one *retained* by a long-lived closure/singleton service (a real leak) — check the retainer chain, not just the presence of detached nodes.
- **Angular DevTools profiler overhead compounds with a deep tree.** On very large component trees, merely having the profiler recording adds measurable overhead per tick, which can make an already-slow interaction look catastrophically worse than it is in the wild — cross-check suspicious absolute numbers against a Chrome Performance trace (no DevTools extension overhead beyond normal V8/Blink instrumentation) before reporting them.
- **Lighthouse CI noise from shared CI runners.** CI containers often share CPU with other jobs, causing run-to-run variance far exceeding what a dedicated machine would show; teams that gate on a single Lighthouse run (`numberOfRuns: 1`) get flaky red builds unrelated to actual regressions — always median multiple runs and prefer relative-regression assertions over absolute thresholds when runner capacity isn't guaranteed.

---

6. Interview Questions & Answers

Q1. What's the difference between Angular DevTools' profiler and Chrome's Performance tab, and when would you reach for each?
Angular DevTools shows you Angular's own model of work — CD cycles, which components were checked, how long each check took, and whether OnPush skipped a subtree. It's framework-aware but blind to everything outside Angular's execution (browser rendering internals, third-party scripts, network). Chrome's Performance tab shows the full picture at the browser level — every JS call stack, style/layout/paint work, long tasks, frame drops — but requires you to manually recognize Angular's own frames (`Zone.runTask`, `ApplicationRef.tick`) inside the flame chart. Use Angular DevTools first to confirm *whether* excess CD is the problem (and which components are involved); use Chrome Performance to see the *full-stack cost* of that CD, correlate it with dropped frames/long tasks, and rule in/out non-Angular causes.

Q2. A component tree of 3,000 rows becomes sluggish when typing in an unrelated filter input elsewhere on the page. Walk through how you'd diagnose it.
First reproduce with a production build and record an Angular DevTools profiling session while typing. Look at the bar-per-keystroke view: if each keystroke produces a tall bar containing checks for all 3,000 rows, that confirms unnecessary CD work rather than, say, a slow API call. Click into one bar to see the per-component breakdown — confirm the row components are default-strategy (not OnPush) or that they are OnPush but receiving a new object/array reference each cycle (defeating the optimization). Cross-check with a Chrome Performance recording to get the actual millisecond cost and see if it crosses the 50ms long-task threshold

(impacting INP). Fix candidates, in order of effort: add `trackBy` to the `*ngFor`, switch rows to `OnPush`, stop re-allocating the filtered array reference on every tick (memoize/compute only on actual input change, or use `computed()` with signals), and consider `@defer`/windowing (virtual scroll via `@angular/cdk/scrolling`) if 3,000 real DOM rows is itself the problem regardless of CD.

Q3. Why can `OnPush` make a bug appear "fixed" in the profiler while actually introducing a stale-data bug?

Interviewer intent: checks that the candidate understands `OnPush` mechanics deeply enough to know it changes semantics, not just performance — a common trap for engineers who apply `OnPush` mechanically after seeing a profiler warning.

`OnPush` only triggers a re-check when: an `@Input()` reference changes (`===` comparison), a DOM event originates inside the component (or a child using it), an `async` pipe emits, or `markForCheck()`/`detectChanges()` is called explicitly. If a parent mutates a bound object or array in place (`this.list.push(item)`) rather than replacing the reference, the `OnPush` child's `@Input()` reference is unchanged, so Angular skips it — the profiler will show fewer/smaller bars (looks like a performance win) but the child's view is now silently out of date. This is why teams pairing `OnPush` with mutable state management get "the button doesn't update" bug reports that never show up as errors — only as an untouched CD bar in the profiler for that component during the mutation.

Q4. Explain LCP, INP, and CLS, and name one Angular-specific practice that improves each.

LCP (Largest Contentful Paint) measures how long until the largest visible element renders; improved in Angular by SSR/hydration (shipping real HTML immediately instead of waiting for CSR) and `NgOptimizedImage` (priority hints, correct sizing) for the hero image. INP (Interaction to Next Paint) measures the worst observed delay between an interaction and the next paint; improved by keeping CD cycles small (OnPush, trackBy, signals, avoiding synchronous heavy work in event handlers) and by adopting zoneless change detection so unrelated async activity doesn't trigger speculative full-tree checks. CLS (Cumulative Layout Shift) measures unexpected layout movement; improved by reserving space for images/embeds (again `NgOptimizedImage` enforces width/height) and avoiding `*ngIf`-driven content insertion above already-rendered content without a placeholder.

Q5. Why might a component look fine in Angular DevTools but still cause a poor Lighthouse/Core Web Vitals score?

Angular DevTools only sees Angular's CD/component layer. A component could have zero unnecessary CD cycles yet still ship an enormous eagerly-loaded chunk (hurting LCP/TBT via parse/compile/execute time before hydration), include an unoptimized image (hurting LCP directly, independent of CD), or trigger CLS via a CSS/font issue that has nothing to do with change detection at all. Lighthouse and Web Vitals evaluate the whole page lifecycle — network, parsing, layout, paint — not just the Angular reactivity layer, so a clean DevTools profile is necessary but not sufficient evidence of good real-world performance.

Q6. How does `trackBy` actually prevent unnecessary DOM work, mechanically?

Without `trackBy`, `NgForOf`'s default differ tracks items **by reference** (or by value for primitives) via Angular's default `IterableDiffer`. If the array reference changes and a new item object exists at index *i* even though it represents "the same" logical entity (same id, updated fields), the differ sees it as removed-old + inserted-new, causing `NgForOf` to destroy the corresponding embedded view (tearing down that row's component instance and DOM nodes — losing local state, re-running lifecycle hooks, re-triggering child CD) and create a fresh one. With `trackBy`, you supply a function returning a stable identity key (e.g., `item.id`); the differ matches old and new arrays by that key, and if an item at the same key still exists, `NgForOf` **reuses** the existing view and just updates its bindings — a cheap property update by rebinding — instead of a destroy+recreate.

Q7. What's the practical difference between profiling in dev mode vs. a production build, and why does it matter for an interview-level "diagnose this" exercise?
Interviewer intent: probes whether the candidate blindly trusts whatever numbers they see, or understands that measurement methodology itself needs validation — a hallmark of senior debugging discipline.
Dev builds run with `ngDevMode` checks enabled (extra assertions, more verbose error paths), unminified and largely un-tree-shaken code, and (if Angular DevTools is attached) additional instrumentation wrapping template functions to produce the profiler's timing data — all of which add real overhead not present for actual users. A component might show 40ms per check in a dev-mode profiler session and 4ms in production. The practical implication: always validate a suspected regression against a production build before prescribing a fix, and always quote absolute timings with the build mode they were captured under — reporting "this component takes 120ms" without specifying dev/prod is professionally meaningless and is exactly the kind of unqualified claim a staff engineer should catch in a review.

Q8. You're asked to add a performance budget to CI so bundle size regressions are caught automatically. Describe the mechanism, not just the tool name.
Two complementary layers: (1) Angular CLI's built-in `budgets` in `angular.json` compare the **build output size** (per bundle type — `initial`, `anyComponentStyle`, etc.) against `maximumWarning`/`maximumError` thresholds at build time, failing the `ng build` step itself if exceeded — cheap, fast, no extra tooling, but coarse (total bytes only, no attribution). (2) Lighthouse CI (`@lhci/cli`) runs full Lighthouse audits against the *served* production build in a CI job, asserting on both bundle-size-adjacent metrics (`total-byte-weight`, `unused-javascript`) and actual runtime metrics (LCP, TBT, CLS) across several runs (to average out CI-runner noise), comparing against a stored baseline so the assertion is "did this regress" rather than an arbitrary absolute cutoff. For deep attribution when a budget trips, run `source-map-explorer` or an esbuild metafile visualizer locally/in an on-demand job to see exactly which module grew.

Q9. A heap snapshot comparison shows a steadily growing number of detached `HTMLDivElement` nodes every time a modal is opened and closed. How do you confirm it's a real leak and find the source?
Take a baseline snapshot, open/close the modal a fixed number of times (say 10) to amplify the signal, force a GC, take a second snapshot, and use Chrome's Comparison view filtered to "Detached" — if the delta count is a multiple of the repetitions (e.g., exactly 10 new detached nodes, or 10 per node type), that's a strong real-leak signal versus noise. Click into one retained object and inspect its **retainers** (the path of references keeping it alive) — commonly this reveals a singleton service holding a `Subscription` to a `Subject`/`Observable` that the modal component subscribed to without unsubscribing, or a closure captured by a `setTimeout`/global event listener registered in `ngOnInit` but never cleared in `ngOnDestroy`. The fix is standard RxJS hygiene: `takeUntilDestroyed()` (Angular 16+) or an explicit `Subscription`/`ngOnDestroy` teardown, or converting to the `async` pipe so Angular manages the subscription lifecycle itself.

Q10. Why is INP considered harder to optimize than LCP for a typical Angular SPA, and what CD-level strategies specifically target it?
Interviewer intent: distinguishes candidates who've only optimized for the "easy," widely-publicized metric (LCP) from those who understand INP requires architectural changes to reactivity, not just asset loading tweaks.
LCP is largely a "one-time, first-load" problem solvable with load-order tricks — preloading, SSR, image priority — that don't require touching your app's ongoing interaction model. INP is measured across **every** interaction for the page's whole lifetime and reports a high percentile (effectively worst-case), so a single slow modal-open handler discovered on page 40 of a session can tank your score even if the initial load was pristine. Because INP is dominated by main-thread blocking during event handling, the actual levers are structural: keeping CD cycles small and targeted (OnPush + immutable state + trackBy so unrelated subtrees aren't rechecked), moving heavy computation off the interaction's critical path (debouncing, `requestIdleCallback`, web

workers), and — most structurally — adopting zoneless change detection or signals so that an unrelated async event (a polling timer, an unrelated HTTP response) doesn't trigger a full-tree CD tick that happens to coincide with, and delay, the user's actual interaction response.

Q11. How would you use the Chrome Performance panel to prove that a "layout thrashing" pattern exists in an Angular component, and how would you fix it?
Record a trace while triggering the suspected code path (e.g., a resize handler or a loop that positions elements). In the flame chart, look for a repeating pattern of a yellow (script) block immediately followed by a narrow purple "Layout" sliver, occurring multiple times within what should be a single logical operation — that repetition is the signature of a read-after-write cycle forcing synchronous reflow on each iteration (e.g., a loop doing `el.style.top = x; const h = el.offsetHeight;` back to back for many elements). Confirm by checking the Bottom-Up view for high self-time under "Recalculate Style"/"Layout" attributed to your component's code. Fix by batching: perform all writes first, then all reads (or vice versa) — read all needed geometry into local variables before mutating any styles — and, in Angular specifically, avoid doing this kind of DOM measurement inside a template getter or an `ngDoCheck`/CD-triggered path at all; move it to `ngAfterViewInit`/`ResizeObserver` callbacks outside the CD cycle, or use `NgZone.runOutsideAngular()` around the measurement code so it doesn't also re-trigger CD on every read/write pair.

Q12. What's the difference between "lab data" and "field data" for Core Web Vitals, and why can't Lighthouse alone tell you your real users' experience?
Lab data (Lighthouse, WebPageTest, a local Performance trace) is synthetic: a fixed device/network/CPU profile running your app in a controlled, repeatable environment — great for CI gating and bisecting regressions because it's deterministic. Field data (Chrome UX Report/CrUX, or your own RUM via the `web-vitals` JS library reporting real users' measurements) reflects the actual distribution of devices, networks, and usage patterns your real audience has — which is what Google's Core Web Vitals assessment (and search ranking signal) is actually based on. A Lighthouse score of 100 can coexist with a poor real-world CrUX assessment if your lab run doesn't reflect your actual traffic's device/network mix (e.g., testing unthrottled on a fast laptop while most users are on mid-tier mobile over 4G) — so lab data should be treated as a regression-detection tool, and field data as the source of truth for whether users are actually having a good experience.

---

7. Quick Revision Cheat Sheet

- **Angular DevTools Profiler** → framework-level view: per-CD-cycle bars, per-component check duration, greyed = skipped (OnPush working). Doesn't see non-Angular work (third-party scripts, layout/paint cost itself).
- **Chrome Performance tab** → ground truth for main-thread cost: flame chart (yellow=JS, purple=style/layout, green=paint/composite), long tasks >50ms flagged, Bottom-Up view for aggregate self-time, Frames track correlates with dropped frames.
- **Zone.js frames** (`Zone.runTask`, `ApplicationRef.tick`) in the flame chart are the bridge that lets you trace a browser-level long task back to a specific Angular event handler/CD cycle.
- **LCP** → largest visible element paint time; levers: SSR/hydration, `NgOptimizedImage`, avoid render-blocking resources.
- **INP** → worst-case interaction-to-paint latency (replaced FID, March 2024); levers: OnPush + trackBy + signals, avoid heavy sync work in handlers, zoneless CD to stop unrelated triggers.
- **CLS** → unexpected layout shift; levers: reserved image/embed dimensions, careful `*ngIf`/placeholder use, font-loading strategy.

- **Lighthouse CI** → lab data, deterministic, use median of multiple runs, gate on relative regression not just absolute score; complements (doesn't replace) field data (CrUX/RUM).
- **Bundle analysis** → `source-map-explorer` (needs source maps), esbuild metafile + visualizer for the modern builder, `angular.json` `budgets` as the first automatic gate; hunt duplicate deps, full-library imports, unused locale data.
- `OnPush` skips a component's check unless: `@Input()` reference changes, event originates inside it, `async` pipe emits, or `markForCheck()` / `detectChanges()` called — mutating bound objects in place silently defeats it (looks like a perf win, is actually a stale-UI bug).
- `trackBy` lets `NgForOf` match old/new array items by stable key, reusing views/DOM instead of destroy+recreate on every reference change.
- **Heap snapshots**: baseline → repeat action N times → force GC → snapshot → Comparison view filtered to growth proportional to N; inspect retainer chains to distinguish transient detached nodes from real leaks (usually unsubscribed `Observable`s or un-torn-down listeners/widgets).
- **Always profile prod builds** for real numbers; dev-mode `ngDevMode` checks + DevTools instrumentation inflate timings. Use CPU/network throttling matching real user conditions, not your dev machine.
- **Don't over-index on one metric** — a great LCP with a terrible INP is still a bad user experience and a failed Core Web Vitals field assessment.
- **Attribute before you fix**: third-party scripts often dominate traces; confirm the cost is actually in your Angular code before rewriting components.

Created By - Durgesh Singh